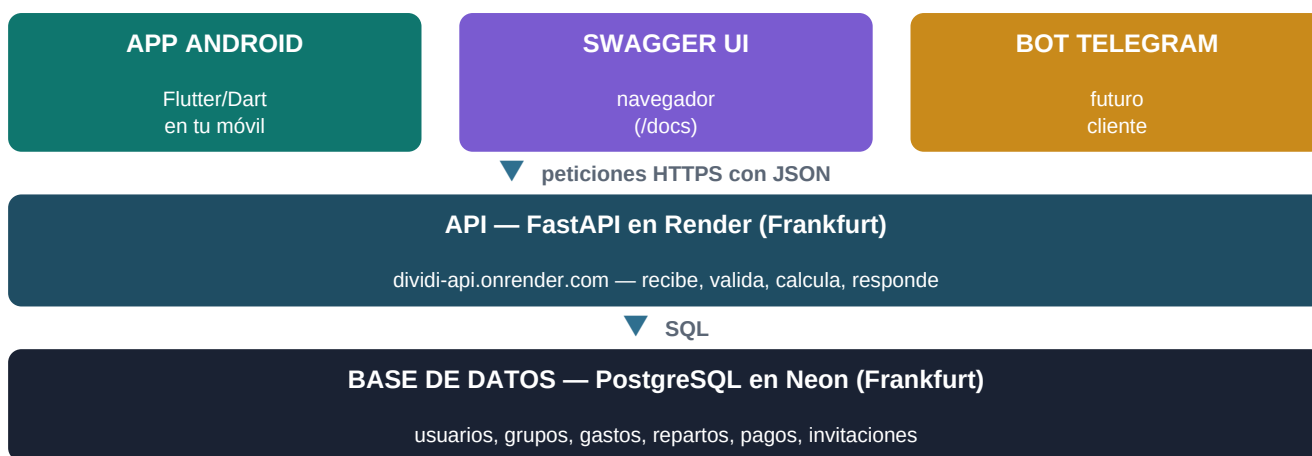


Dividi — Arquitectura del sistema

La app Android, la API en la nube y cómo interactúan, pieza a pieza

Dividi es un sistema completo de gastos compartidos (tipo Splitwise) formado por **tres piezas que viven en sitios distintos**: una app Android, una API en un servidor en la nube y una base de datos gestionada. Este documento explica qué hace cada pieza, por qué está diseñada así y, sobre todo, cómo se hablan entre ellas. Cada término técnico se define la primera vez que aparece; al final hay un glosario completo.

1 El mapa general: tres piezas, tres lugares



La regla de oro del diseño: **toda la inteligencia vive en la API**. La app no calcula repartos ni balances, no valida porcentajes contra la base de datos, no decide quién puede borrar qué. Solo pinta pantallas, recoge lo que escribes y se lo manda a la API. Por eso el mismo backend sirve a la vez a la app, a Swagger y a cualquier cliente futuro sin cambiar una línea: todos son «caras» distintas del mismo cerebro.

API REST / Endpoint

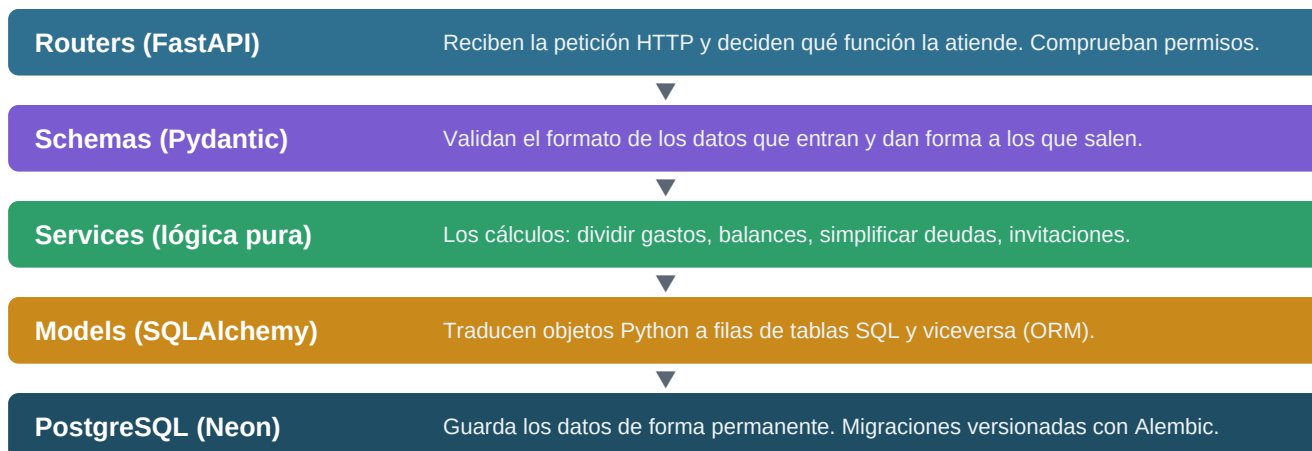
La API es el conjunto de operaciones que el backend ofrece por internet. Cada operación concreta es un **endpoint**: un verbo HTTP (GET leer, POST crear, PATCH editar, DELETE borrar) más una dirección. Ej.: `POST /groups/123/expenses` crea un gasto en el grupo 123. Los datos viajan en **JSON**, texto con pares campo:valor que cualquier lenguaje entiende — por eso una app en Dart habla sin problema con un servidor en Python.

Por qué separar app y backend

Si la lógica viviera en la app, cada usuario tendría su propia copia de los datos y las cuentas del grupo no cuadrarían entre móviles. Con un backend central, todos ven lo mismo, las reglas se aplican una sola vez y en un solo sitio, y añadir una nueva plataforma (iPhone, web, bot) no requiere reescribir nada del cerebro.

2 La API por dentro (repositorio dividi)

El backend está escrito en **Python** con **FastAPI** y organizado en capas: cada petición las atraviesa de arriba abajo. Cada capa tiene una única responsabilidad, lo que permite probar y cambiar una sin romper las demás.



Qué ofrece la API, por dominios

Dominio	Endpoints	Qué resuelven
Auth	register / login / refresh	Alta con código de invitación, tokens JWT de sesión
Invitaciones	crear / listar / validar / revocar	Acceso invite-only: códigos de un solo uso, con caducidad o email opcionales
Grupos	CRUD + miembros	Grupos con miembros (con o sin cuenta) y porcentaje por defecto que siempre suma 100
Gastos	CRUD + filtros	4 métodos de reparto; al editar se recalcula todo
Balances	balances / settle-up	Cuánto debe o le deben a cada uno, y los pagos mínimos para saldar
Pagos	crear / listar	Registrar que alguien devolvió dinero a otro

El modelo de datos: qué guarda cada tabla

Tabla	Qué representa	Relación clave
users	Cuentas registradas (email, hash de contraseña, nombre)	—
groups	Cada lista de gastos: nombre, moneda, dueño	pertenece a un user
group_members	Cada persona dentro de un grupo, con o sin cuenta, su rol (admin/member) y su % por defecto	une users y groups; user_id puede ser nulo (invitado)
expenses	Un gasto: descripción, importe, categoría, quién pagó, método de reparto	pertenece a un group
expense_splits	La parte de cada participante en un gasto, con su computed_amount final	une expenses y group_members
payments	Un pago real entre dos personas para saldar deudas (no es un gasto)	dos group_members: de quién a quién
invitations	Códigos de acceso a la plataforma: quién los creó, quién los usó, caducidad	creada y usada por users

Las tres piezas de lógica que hacen especial este backend

Pieza	Qué hace	Detalle clave
split_calculator	Divide un gasto entre participantes según el método: equal, percentage, exact o shares	Redondeo a 2 decimales; el último participante absorbe la diferencia para que la suma sea exacta
balance_service	Balance neto de cada miembro: lo aportado menos lo consumido	aportado = gastos adelantados + pagos hechos; consumido = su parte de cada gasto + pagos recibidos
debt_simplifier	Calcula los pagos mínimos para saldar el grupo (settle-up)	Algoritmo greedy sobre heaps: empareja al mayor deudor con el mayor acreedor. Máximo $n-1$ transacciones, $O(n \log n)$

Invite-only y el fundador

Registrarse exige un código de invitación válido... con una excepción: el **primer usuario** de la base de datos (el fundador) entra sin código. Cada código se consume al usarse (un solo uso), puede atarse a un email concreto y puede caducar. Así la URL pública no significa registro abierto: la puerta es visible, pero solo entra quien tiene llave — y generar más llaves cuesta un toque.

Miembros invitados sin cuenta

Un miembro de grupo puede existir solo con un nombre («Persona 1», «Compi»), sin usuario registrado detrás. Sirve para llevar cuentas sin que todos usen la app, o para cuentas individuales. Si se le asocia un email y esa persona se registra algún día con él, su cuenta se vincula automáticamente a todas sus membresías pendientes.

3 La app por dentro (repositorio dividi-app)

La app está hecha con **Flutter**, el framework de Google, y escrita en **Dart**. Motivo de la elección: un único código compila a app nativa de Android y de iPhone; Kotlin (Android) o Swift (iPhone) habrían exigido dos proyectos separados. Todo el código propio vive en la carpeta `lib/`; las carpetas `android/` e `ios/` son solo el envoltorio nativo que Flutter gestiona.

Archivo	Responsabilidad
main.dart	Arranque. Comprueba si hay sesión guardada: con sesión va directo a Mis grupos; sin ella, al login
services/api_client.dart	La única puerta a la API: todas las peticiones HTTP, el guardado cifrado de tokens y su renovación automática
screens/login_screen.dart	Formulario de email + contraseña; llama a POST /auth/login
screens/register_screen.dart	Alta con nombre, email, contraseña y código de invitación
screens/groups_screen.dart	Lista de grupos del usuario, crear grupo, cerrar sesión
screens/group_detail_screen.dart	Detalle de un grupo con 3 pestañas: Gastos, Balances y Settle up
screens/expense_form_screen.dart	Crear y editar gastos: participantes, método de reparto, porcentajes inteligentes, borrar
screens/members_screen.dart	Miembros del grupo y añadir nuevos (invitados sin cuenta o por email)

Widget y estado (los ladrillos de Flutter)

En Flutter todo lo que ves es un **widget**: un botón, un campo de texto, una pantalla entera. Las pantallas que cambian con el tiempo (cargando... → datos) son **StatefulWidget**: guardan un estado y, cuando este cambia (con `setState`), Flutter redibuja solo lo necesario. Así funciona, por ejemplo, la ruedita de carga del botón al crear un gasto, o los porcentajes que se reajustan al escribir.

Asíncrono (`async/await` y `Future`)

Hablar con internet tarda (milisegundos... o 50 segundos si el servidor duerme). Para no congelar la pantalla mientras tanto, las llamadas devuelven un **Future**: una promesa de resultado. La palabra `await` pausa esa función (y solo esa) hasta que llegue la respuesta, mientras la interfaz sigue viva. Es el equivalente Dart del `async` de Python que usa la propia API.

Los porcentajes inteligentes del formulario de gasto

El formulario no te obliga a cuadrar los porcentajes a mano. Cada campo está en uno de dos modos: **fijado** (escribiste un valor; icono de candado) o **automático** (icono de flechas). Lo que falta hasta 100 se reparte a partes iguales entre los automáticos, en vivo: con 3 personas sale 33.33 / 33.33 / 33.34; escribes 50 en una y las otras pasan solas a 25 / 25; borras un campo y vuelve al modo automático. La app valida la suma antes de enviar y la API la revalida después: la del cliente es comodidad, la del servidor es ley (una app trucada no puede saltarse las reglas).

4 Cómo se hablan: la sesión y los tokens

La API es **sin estado**: no recuerda quién eres entre peticiones. Cada petición debe demostrar la identidad con un token. El ciclo completo:

1

Login

La app envía email y contraseña a POST /auth/login. La API verifica el hash bcrypt y responde con dos tokens JWT.

2

Dos tokens, dos vidas

access token (30 minutos): se adjunta a cada petición en la cabecera Authorization: Bearer. refresh token (7 días): solo sirve para pedir tokens nuevos.

3

Guardado cifrado

La app los guarda con flutter_secure_storage (Keystore de Android): sobreviven a cerrar la app, y ni otras apps ni un backup simple pueden leerlos.

4

Renovación automática

Si una petición devuelve 401 (token caducado), el api_client llama solo a POST /auth/refresh, guarda los tokens nuevos y reintenta la petición original. Tú no ves nada.

5

Cierre de sesión honesto

Solo si el refresh devuelve un 401 real (los 7 días expiraron) se borra la sesión y vuelves al login. Un error transitorio del servidor (p.ej. despertando) no te expulsa.

JWT (JSON Web Token)

Un texto firmado criptográficamente por el servidor que contiene quién eres (tu id de usuario) y hasta cuándo vale. La firma usa la SECRET_KEY del servidor: nadie puede fabricar ni alterar un token sin ella, y el servidor puede verificarlo sin consultar la base de datos. Analogía: la pulsera de un festival — te la dan al entrar y basta enseñarla en cada puerta.

5 Un gasto de punta a punta (el recorrido completo)

Ejemplo real: en el grupo Piso (Diego y Compi al 50/50), Diego crea una cena de 100 € repartida 70/30. Esto es todo lo que ocurre:

1

En la app: formulario

Diego rellena descripción e importe, elige Porcentajes, escribe 70 en su campo; el de Compi pasa solo a 30. La app valida en local: suma 100, importe > 0.

2

La app construye el JSON

api_client.createExpense() monta {description, amount, paid_by, split_method: percentage, splits: [{member, 70}, {member, 30}]} y lo envía por HTTPS con el access token en la cabecera.

3

FastAPI recibe y Pydantic valida

¿El JSON tiene la forma correcta? ¿Tipos válidos, importe positivo? Si no, 422 automático sin ejecutar nada más.

4

Autenticación y permisos

Se verifica la firma del JWT, se carga el usuario, y se comprueba que es miembro del grupo y que paid_by pertenece al grupo.

5

split_calculator calcula

70% de 100 = 70.00 para Diego; 30.00 para Compi. Valida que los porcentajes sumen 100 (regla del servidor, no negociable). Guarda cada parte como computed_amount.

6

SQLAlchemy escribe en Neon

Una fila en expenses y dos en expense_splits, dentro de una transacción: o se guarda todo o no se guarda nada.

7

Respuesta 201 y refresco

La API devuelve el gasto completo en JSON. La app cierra el formulario y recarga gastos, balances y settle-up: Diego +30, Compi -30; sugerencia: Compi paga 30 a Diego.

Los errores hacen el viaje inverso: si algo falla, la API responde con un código (400 regla de negocio violada, 401 sin identificar, 403 sin permiso, 404 no existe, 422 formato inválido) y un campo `detail` explicando el motivo. El `api_client` lo convierte en una `ApiException` y la pantalla lo muestra en un aviso — por eso ves mensajes como «Los porcentajes del gasto deben sumar 100» escritos por el backend.

6 Dónde vive cada cosa: la infraestructura

Pieza	Servicio	Papel y detalles
API	Render (plan gratuito, Frankfurt)	Ejecuta el contenedor Docker de la API 24/7. Con inactividad se duerme: la primera petición tarda hasta ~50 s en despertarlo (por eso la app muestra ruedas de carga y bloquea el botón para evitar envíos dobles)
Base de datos	Neon (PostgreSQL serverless, Frankfurt)	Guarda todos los datos. Independiente de la API: aunque Render duerma o se redesplice, los datos no se tocan. Misma región que la API para mínima latencia
Código	GitHub (2 repos públicos)	dividi (backend) y dividi-app (app). Render redesplica solo con cada push a main del backend
Secretos	Variables de entorno en Render	DATABASE_URL (conexión a Neon), SECRET_KEY (firma de los JWT), REQUIRE_INVITE. Nunca en el código ni en los repos

Docker y el despliegue

La API se empaqueta en un **contenedor**: una caja con el código, Python y todas las dependencias, que corre idéntica en cualquier máquina. El mismo Dockerfile sirve para desarrollo local y para Render. Al arrancar, el contenedor aplica primero las **migraciones** de Alembic (cambios versionados del esquema de la base de datos) y luego levanta el servidor — así el código y las tablas nunca se desincronizan.

Los tests como red de seguridad

El backend tiene 86 tests (lógica de reparto, deudas, invitaciones, permisos, API completa) que corren sobre una base de datos efímera en memoria. La app tiene tests de widgets para el login y para el sistema de porcentajes automáticos. Antes de cada cambio importante se ejecutan: si algo se rompe, se sabe al momento y no en el móvil de un amigo.

7 Las decisiones de diseño y su porqué

Decisión	Por qué
Flutter/Dart y no Kotlin	Un solo código para Android e iPhone. Kotlin habría atado la app a Android y duplicado el trabajo futuro
Toda la lógica en la API	Una sola fuente de verdad: las reglas se aplican igual para cualquier cliente y no se pueden esquivar trucando la app
JWT sin estado y no sesiones	El servidor no guarda sesiones: escala horizontalmente y cualquier instancia puede atender cualquier petición

Decisión	Por qué
Access corto + refresh largo	Si roban un access token, caduca en 30 min. El refresh, más valioso, viaja mucho menos (solo al renovar)
Decimal para el dinero, jamás float	Los floats acumulan errores de redondeo; con dinero eso es inaceptable. Toda la cadena usa decimales exactos
computed_amount precalculado	El reparto se calcula una vez al escribir el gasto, no en cada consulta de balances: lecturas rápidas y consistentes
Greedy en settle-up	El óptimo absoluto es NP-hard; el greedy garantiza como mucho $n-1$ pagos en $O(n \log n)$ y es lo que usan las apps reales
Invite-only con fundador	Grupo privado de amigos que puede crecer sin límite: generar una invitación cuesta un toque
Invitados sin cuenta	Puedes llevar las cuentas del piso aunque nadie más instale la app: los demás son nombres hasta que quieran registrarse
Validar en cliente Y en servidor	La validación de la app es experiencia de usuario (feedback inmediato); la del servidor es seguridad (nadie puede saltarla)

8 Glosario

Backend / Frontend El cerebro invisible que procesa y guarda (backend) y la cara visible con la que interactúas (frontend). Aquí: API y app.

API REST Conjunto de operaciones que el backend ofrece por internet, organizadas como verbos HTTP + direcciones.

Endpoint Una operación concreta de la API. Ej.: GET /groups/{id}/balances.

HTTP / HTTPS El protocolo de petición/respuesta de internet; la S añade cifrado del tráfico.

JSON Formato de texto con pares campo:valor en el que viajan los datos entre app y API.

JWT Token firmado que demuestra tu identidad en cada petición sin reenviar la contraseña.

Access / refresh token El primero (30 min) acompaña cada petición; el segundo (7 días) solo sirve para renovar al primero.

bcrypt / hash Transformación irreversible de la contraseña: el servidor guarda el hash, nunca la contraseña.

FastAPI Framework de Python que convierte funciones en endpoints y genera la documentación (/docs) automáticamente.

Pydantic / Schema Capa que valida que los datos recibidos tengan la forma y tipos correctos antes de ejecutar nada.

ORM (SQLAlchemy) Traductor automático entre objetos del código y filas de tablas SQL.

Migración (Alembic) Cambio versionado del esquema de la base de datos, aplicado en orden en cualquier entorno.

PostgreSQL / Neon La base de datos relacional del proyecto y el servicio en la nube que la aloja (serverless, Frankfurt).

Render El servicio en la nube que ejecuta el contenedor de la API 24/7 (con siesta en el plan gratis).

Docker / contenedor Caja que empaqueta la app con todo lo necesario para correr igual en cualquier máquina.

Flutter / Dart Framework de Google y su lenguaje: un código único que compila a app nativa de Android e iPhone.

Widget Cada pieza de interfaz en Flutter: botón, campo, pantalla. Se componen unos dentro de otros.

StatefulWidget / setState Widget con memoria interna; setState avisa a Flutter de que algo cambió para redibujar.

Future / async / await Mecánica de Dart para esperar respuestas de internet sin congelar la pantalla.

flutter_secure_storage Almacén cifrado del dispositivo (Keystore) donde la app guarda los tokens.

APK El paquete instalable de una app Android; lo que produce flutter build apk.

Emulador Móvil Android virtual corriendo en el ordenador para probar la app sin dispositivo físico.

Swagger UI Página interactiva en /docs para probar cada endpoint de la API desde el navegador.

Settle-up El cálculo de los pagos mínimos que dejan a todo el grupo en paz (balances a cero).

Invite-only / fundador Registro solo con código de invitación; el primer usuario del sistema entra sin él.

Serverless / sleep Servicios que se apagan solos sin uso (y cobran/consumen solo al trabajar); despertar tiene un pequeño coste de espera.